



《AI大模型Ragent项目》—— 单次提问同时查三个知识库——多通道并行检索架构

来自： 拿个offer-开源&项目实战



马丁

2026年05月14日 21:55

开篇引言

上一篇讲完了意图分数到检索动作的映射——SYSTEM 意图短路直接回复，MCP 意图触发工具调用，KB 意图走定向检索或全局兜底。四个电商客服场景跑了一遍，每种意图类型各走一条路径。读者已经知道哪些通道会被激活、每个通道查多少条。用一句话概括第 9 篇和本篇的分工：第 9 篇解决的是查什么——哪些通道该激活、查哪个 Collection、TopK 多少。本篇解决的是怎么查——通道怎么并行跑、线程池怎么隔离、结果怎么收回来、挂了怎么容错。来看一个具体场景。假设用户在电商客服界面发了一句：

AirPods 保修多久，顺便退货运费谁出？

经过查询重写拆成两个子问题，意图分类分别命中了 3C 数码 > 保修政策 (score=0.88) 和 3C 数码 > 退货政策 (score=0.85)。第 9 篇告诉我们，定向检索通道被激活了，要查 kb_3c_warranty 和 kb_3c_return 两个 Milvus Collection。分数都在 0.85 以上，全局兜底通道不启用。两个 Collection 的向量检索各花 50ms 左右。如果串行查，合计 100ms。并行查，50ms 就出结果。快了一倍听起来不错，但问题不只是快不快——

- 通道之间用什么线程池？
- 通道内部多个 Collection 又用什么线程池？会不会互相抢资源？
- 如果 kb_3c_warranty 对应的 Milvus 分区正在 compaction 导致超时了怎么办？

这就是多通道并行检索 (Multi-Channel Parallel Retrieval) 要解决的问题。

总览：MultiChannelRetrievalEngine 的两阶段架构

1. 入口方法

MultiChannelRetrievalEngine 是多通道检索的核心编排者。它的入口方法 retrieveKnowledgeChannels 把整个检索过程分成两个阶段：

```
@RagTraceNode(name = "multi-channel-retrieval", type = "RETRIEVE_CHANNEL")
public List<RetrievedChunk> retrieveKnowledgeChannels(List<SubQuestionIntent> subIntents, int topK)
// 构建检索上下文
SearchContext context = buildSearchContext(subIntents, topK);

// 【阶段1：多通道并行检索】
List<SearchChannelResult> channelResults = executeSearchChannels(context);
if (CollUtil.isEmpty(channelResults)) {
    return List.of();
}

// 【阶段2：后置处理器链】
return executePostProcessors(channelResults, context);
}
```

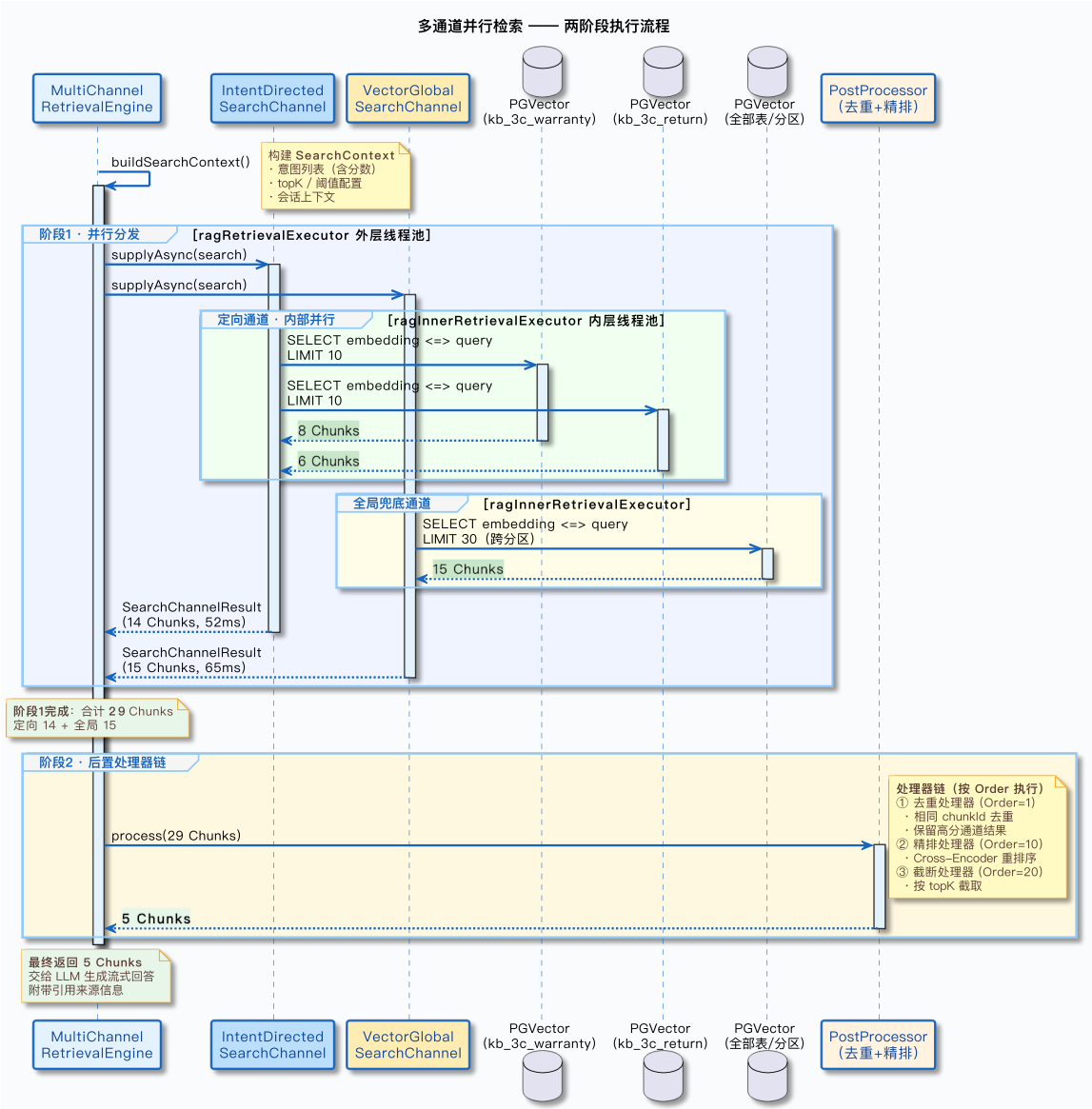
两阶段的职责很清楚：

- 阶段 1：executeSearchChannels——并行执行所有启用的 Channel，收集每个通道的原始结果
- 阶段 2：executePostProcessors——串行执行后处理器链，做去重、精排、截断

本篇聚焦阶段 1。阶段 2 的后处理器链 (去重 + Cross-Encoder 精排) 在第 11 篇详细展开，这里只需要知道它的输入是阶段 1 收回来的所有 Chunk，输出是精选后的最终 Chunk 列表。

2. 两阶段的执行流程

为了展示完整的两阶段流程，下面这张时序图以两个通道同时启用为例 (比如用户问题的意图分数在 0.55 左右，定向和全局通道都被激活)，展示从入口到向量数据库再到后处理的执行过程：



图里有几个关键细节：

- 两个 Channel 在外层线程池 (ragRetrievalExecutor) 里并行执行，耗时取决于最慢那个
- 定向检索通道内部又在内层线程池 (ragInnerRetrievalExecutor) 里并行查两个 Collection
- 所有原始结果汇聚后，串行经过后置处理器链，最终输出精选结果

3. SearchContext：所有通道共享的上下文

所有通道拿到的是同一个 SearchContext 对象：

```
@Data
@Builder
public class SearchContext {

    private String originalQuestion;    // 原始问题
    private String rewrittenQuestion;   // 改写后的问题
    private List<String> subQuestions;  // 子问题列表
    private List<SubQuestionIntent> intents; // 意图识别结果
    private int topK;                  // 期望返回的结果数量

    @Builder.Default
    private Map<String, Object> metadata = new HashMap<>();

    // 优先使用改写后的问题做检索
    public String getMainQuestion() {
        return rewrittenQuestion != null ? rewrittenQuestion : originalQuestion;
    }
}
```

getMainQuestion() 优先用改写后的问题做检索——第 4 篇查询重写把那它保修多久改写成了 AirPods Pro 的保修期是多久，检索用改写后的版本效果更好。

4. SearchChannelResult：通道的输出

每个通道返回一个 SearchChannelResult：

```
@Data
@Builder
public class SearchChannelResult {

    private SearchChannelType channelType; // 通道类型标识
    private String channelName;           // 通道名称
    private List<RetrievedChunk> chunks;  // 检索到的 Chunk 列表
    private long latencyMs;               // 检索耗时（毫秒）

    @Builder.Default
    private Map<String, Object> metadata = new HashMap<>();
}
```

channelType 不只是打日志用——后处理器做去重时会根据这个字段判断优先级。同一个 Chunk 被定向检索（INTENT_DIRECTED）和全局检索（VECTOR_GLOBAL）同时命中时，保留定向检索的版本，因为定向检索是基于意图识别的精准匹配，可信度更高。latencyMs 则方便排查哪个通道拖了后腿。

SearchChannel 接口：通道的统一契约

1. 五个方法各干什么

SearchChannel 是所有检索通道的统一接口：

```
public interface SearchChannel {

    /** 通道名称（用于日志和监控） */
    String getName();

    /** 通道优先级（数字越小优先级越高） */
    int getPriority();

    /** 是否启用该通道 */
    boolean isEnabled(SearchContext context);

    /** 执行检索 */
}
```

```
SearchChannelResult search(SearchContext context);

/** 通道类型 */
SearchChannelType getType();
}
```

五个方法各司其职：

方法	职责	举例
getName()	日志打印和监控标识	IntentDirectedSearch
getPriority()	去重时的优先级，数字越小越优先	定向=1，全局=10
isEnabled(context)	根据当前请求动态判断是否启用	第 9 篇已详细讲过
search(context)	执行实际检索，返回 Chunk 列表	本篇重点
getType()	标识通道类型，后处理器据此判断来源	INTENT_DIRECTED

isEnabled 接收 SearchContext 而不是无参方法，这个设计很关键——同一个 Channel 对不同请求的表现可以完全不同。用户问 AirPods 保修多久时定向检索启用、全局检索不启用；用户含糊地问售后怎么弄时两个通道同时启用。激活条件的具体逻辑在第 9 篇已经拆过，这里不重复。

2. 接口驱动的扩展性

MultiChannelRetrievalEngine 通过 Spring 的列表注入拿到所有通道实现：

```
private final List<SearchChannel> searchChannels;
```

Spring 会自动收集所有实现了 SearchChannel 接口的 @Component，注入到这个列表里。这意味着新增一个检索通道的步骤是：

- 1. 写一个新类实现 SearchChannel
- 2. 加上 @Component 注解
- 3. 搞定——主引擎代码零改动，新通道自动生效

SearchChannelType 枚举也为未来扩展留了位置：

```
public enum SearchChannelType {
    VECTOR_GLOBAL,      // 当前实现
    INTENT_DIRECTED,    // 当前实现
    KEYWORD_ES,         // 预留：ES 关键词检索
    HYBRID               // 预留：混合检索
}
```

当前只有 INTENT_DIRECTED 和 VECTOR_GLOBAL 两种实现，但架构上随时可以加 ES 关键词检索或混合检索通道。

3. 过滤、排序与并行执行

executeSearchChannels 里的编排逻辑值得看一下：

```
private List<SearchChannelResult> executeSearchChannels(SearchContext context) {
    // 过滤启用的通道，按优先级排序
    List<SearchChannel> enabledChannels = searchChannels.stream()
        .filter(channel -> channel.isEnabled(context))
        .sorted(Comparator.comparingInt(SearchChannel::getPriority))
        .toList();

    if (enabledChannels.isEmpty()) {
        return List.of();
    }

    // 并行提交到外层线程池
```

```
List<CompletableFuture<SearchChannelResult>> futures = enabledChannels.stream()
    .map(channel -> CompletableFuture.supplyAsync(
        () -> {
            try {
                return channel.search(context);
            } catch (Exception e) {
                log.error("检索通道 {} 执行失败", channel.getName(), e);
                return emptyResult(channel);
            }
        },
        ragRetrievalExecutor
    ))
    .toList();

// 等待所有通道完成
List<SearchChannelResult> results = futures.stream()
    .map(CompletableFuture::join)
    .filter(Objects::nonNull)
    .toList();

return results;
}
```

这段代码有个容易被忽略的点：虽然 `enabledChannels` 按 `getPriority()` 排了序，但所有通道都是通过 `CompletableFuture.supplyAsync` 并行执行的——排序不影响执行顺序，只影响日志打印和结果列表的稳定性。

还有一个关键设计——每个 `Channel` 的 `search()` 被 `try-catch` 包裹，失败时返回 `emptyResult`（空 `Chunk` 列表）而不是抛异常。这保证了单个通道崩溃不会拖垮整个检索。

两个通道各自怎么搜

1. IntentDirectedSearchChannel：按意图并行查多个 Collection

1.1 内部执行流程

定向检索通道的优先级是 1（最高），它的 `search()` 方法做的事情很明确——把 KB 意图列表交给 `IntentParallelRetriever`，对每个意图绑定的 `Collection` 做并行检索：

```
@Override
public SearchChannelResult search(SearchContext context) {
    long startTime = System.currentTimeMillis();

    try {
        List<NodeScore> kbIntents = extractKbIntents(context);

        // 并行检索所有意图对应的知识库
        int topKMultiplier = properties.getChannels().getIntentDirected().getTopKMultiplier();
        List<RetrievedChunk> allChunks = parallelRetriever.executeParallelRetrieval(
            context.getMainQuestion(),
            kbIntents,
            context.getTopK(),
            topKMultiplier
        );

        long latency = System.currentTimeMillis() - startTime;

        return SearchChannelResult.builder()
            .channelType(SearchChannelType.INTENT_DIRECTED)
            .channelName(getName())
            .chunks(allChunks)
            .latencyMs(latency)
            .metadata(Map.of("intentCount", kbIntents.size()))
            .build();
    }
```

```
} catch (Exception e) {
    log.error("意图定向检索失败", e);
    return SearchChannelResult.builder()
        .channelType(SearchChannelType.INTENT_DIRECTED)
        .channelName(getName())
        .chunks(List.of())
        .latencyMs(System.currentTimeMillis() - startTime)
        .build();
}
```

流程很清晰：提取 KB 意图 → 委托并行检索器 → 包装成 SearchChannelResult 返回。整个 search() 外面包了 try-catch——即使 parallelRetriever 内部出了意料之外的异常，也返回空结果而不是让调用方崩溃。

1.2 构造器注入内层线程池

注意构造器里的一个细节：

```
public IntentDirectedSearchChannel(RetrieverService retrieverService,
                                   SearchChannelProperties properties,
                                   @Qualifier("ragInnerRetrievalThreadPoolExecutor") Executor ragI
    this.properties = properties;
    this.parallelRetriever = new IntentParallelRetriever(retrieverService, ragInnerRetrievalExecut
}
```

ragInnerRetrievalThreadPoolExecutor 是内层线程池，和外层的 ragRetrievalExecutor 不同。IntentParallelRetriever 在构造时就创建好了，不会每次 search() 时重新创建——检索器是无状态的，复用同一个实例就行。

2. VectorGlobalSearchChannel：扫全部 Collection

2.1 Collection 列表从哪来

全局检索通道的优先级是 10（最低），它做的事情更粗暴——从数据库查出所有知识库的 Collection，全部扫一遍：

```
private List<String> getAllKBCollections() {
    Set<String> collections = new HashSet<>();

    List<KnowledgeBaseDO> kbList = knowledgeBaseMapper.selectList(
        Wrappers.lambdaQuery(KnowledgeBaseDO.class)
            .select(KnowledgeBaseDO::getCollectionName)
            .eq(KnowledgeBaseDO::getDeleted, 0)
    );
    for (KnowledgeBaseDO kb : kbList) {
        String collectionName = kb.getCollectionName();
        if (collectionName != null && !collectionName.isBlank()) {
            collections.add(collectionName);
        }
    }

    return new ArrayList<>(collections);
}
```

从 t_knowledge_base 表查出所有未删除的知识库的 collectionName，用 HashSet 去重（同一个 Collection 可能被多条记录引用），然后交给 CollectionParallelRetriever 并行检索。

每次全局检索都会查一次数据库拿 Collection 列表。知识库变更不频繁的话，可以考虑加 Redis 缓存（类似第 5 篇意图树的缓存策略）。当前实现选择了简单可靠——查数据库的开销是毫秒级的，相对于向量数据库检索的几十毫秒可以接受。

2.2 topK 的处理差异

全局检索的 topK 计算比定向检索简单得多——所有 Collection 用同一个值：

```
int topKMultiplier = properties.getChannels().getVectorGlobal().getTopKMultiplier();
List<RetrievedChunk> allChunks = parallelRetriever.executeParallelRetrieval(
    context.getMainQuestion(),
    collections,
    context.getTopK() * topKMultiplier // 直接乘以倍率
);
```

定向检索每个意图有自己独立的 topK（第 9 篇讲过，从意图节点配置取），全局检索不关联具体意图节点，统一用 context.getTopK() * topKMultiplier。

3. 两个通道的对比

把两个通道的关键差异放到一张表里：

维度	IntentDirectedSearchChannel	VectorGlobalSearchChannel
优先级	1（最高）	10（较低）
检索范围	意图绑定的 Collection	数据库中全部 Collection
TopK 计算	每个意图独立计算（节点配置 → fallback → × 倍率）	统一 topK × 倍率
topKMultiplier	2	3（噪音更大，需要更多候选）
并行检索器	IntentParallelRetriever（泛型 IntentTask）	CollectionParallelRetriever（泛型 String）
Collection 来源	意图节点的 collectionName	数据库 t_knowledge_base 表

为什么全局检索的 topKMultiplier=3 比定向检索的 2 大？定向检索的 Collection 是意图识别精准匹配的结果，召回质量相对高，2 倍的候选量够用。全局检索扫全库，和用户问题不相关的 Collection 也在搜，噪声大，需要更多候选给后续 Reranker 筛选。

并行执行：两级线程池隔离

1. 为什么需要两级线程池

先看一下整个检索过程的并行结构：



两层并行嵌套——外层是 Channel 之间的并行，内层是单个 Channel 内部多个 Collection 之间的并行。如果两层共用同一个线程池会怎样？假设线程池最大 4 个线程，两个 Channel 各占一个线程跑 search()。search() 内部又要提交 Collection 检索任务到同一个线程池——但池子里只剩 2 个线程了。如果全局检索要查 5 个 Collection，前 2 个占了剩余线程，后 3 个排队等。外层的两个线程在 .join() 等内层完成，内层在等外层释放线程——形成类似死锁的拥塞。分开就没这问题：外层线程池只管 Channel 级并行，内层线程池只管 Collection 级并行，两个池子互不干扰。

2. 外层线程池：ragRetrievalThreadPoolExecutor

```
@Bean
public Executor ragRetrievalThreadPoolExecutor() {
    ThreadPoolExecutor executor = new ThreadPoolExecutor(
```

```
        CPU_COUNT,                // 核心线程数 = CPU 核数
        CPU_COUNT << 1,           // 最大线程数 = 2 倍 CPU
        60, TimeUnit.SECONDS,
        new SynchronousQueue<>(), // 无缓冲直接交付
        ThreadFactoryBuilder.create()
            .setNamePrefix("rag_retrieval_executor_")
            .build(),
        new ThreadPoolExecutor.CallerRunsPolicy()
    );
    return TtlExecutors.getTtlExecutor(executor);
}
```

关键参数：

- 核心线程 = CPU 核数，最大 = 2 倍 CPU：Channel 数量固定（当前 2 个）
- SynchronousQueue（同步队列）：无缓冲队列，来任务直接交给线程，没有空闲线程就创建新的（直到最大值）。Channel 数量少，不需要排队，直接交付延迟最低
- CallerRunsPolicy（调用方执行策略）：拒绝策略——如果所有线程都忙且队列满了，提交任务的线程自己执行该任务。对于外层线程池，极端情况下退化为串行执行 Channel，但不会丢弃任务

3. 内层线程池：ragInnerRetrievalThreadPoolExecutor

```
@Bean
public Executor ragInnerRetrievalThreadPoolExecutor() {
    ThreadPoolExecutor executor = new ThreadPoolExecutor(
        CPU_COUNT << 1,           // 核心线程数 = 2 倍 CPU
        CPU_COUNT << 2,           // 最大线程数 = 4 倍 CPU
        60, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(100), // 有缓冲排队
        ThreadFactoryBuilder.create()
            .setNamePrefix("rag_inner_retrieval_executor_")
            .build(),
        new ThreadPoolExecutor.CallerRunsPolicy()
    );
    return TtlExecutors.getTtlExecutor(executor);
}
```

和外层线程池的差异：

参数	外层 (ragRetrievalExecutor)	内层 (ragInnerRetrievalExecutor)
核心线程	CPU 核数	2 倍 CPU
最大线程	2 倍 CPU	4 倍 CPU
队列	SynchronousQueue（无缓冲）	LinkedBlockingQueue(100)
服务对象	Channel 级并行（2 个）	Collection 级并行（可能 10+）

内层为什么比外层大？外层的并发度是 Channel 数量（目前 2 个），内层的并发度是 Collection 数量——全局检索可能同时查 10~20 个 Collection，向量检索是 IO 密集型（主要是网络等待），CPU 利用率不高，可以承载更多并发。LinkedBlockingQueue(100)（有界阻塞队列）提供了排队缓冲，队列长度 100 足够覆盖任何合理数量的 Collection。两个线程池都用了 CallerRunsPolicy 拒绝策略，保证极端情况下退化为串行而不是丢任务。

4. TTL 透传

两个线程池都用 TtlExecutors.getTtlExecutor() 包装。TTL 是阿里的 TransmittableThreadLocal 工具——普通的 ThreadLocal 在线程池场景下会丢失上下文（父线程设的值子线程拿不到），TTL 能把 ThreadLocal 的值自动透传到子线程。没有 TTL 包装的话，内层线程池里的向量检索任务拿不到链路追踪的 traceId，日志关联就断了——你在日志里看到一条检索耗时异常的记录，却不知道它属于哪个用户的哪次请求。

AbstractParallelRetriever：模板方法模式

1. 为什么抽出父类

定向检索通道内部要对多个意图做并行检索，全局检索通道内部要对多个 Collection 做并行检索。两边的并行执行逻辑一模一样——创建 Future、提交到线程池、等待完成、收集结果、统计日志。不同的只是具体检索什么和日志里怎么标识检索目标。

经典的模板方法模式应用场景：把不变的骨架逻辑放在父类，把可变的部分留给子类实现。

2. 父类的骨架方法

```

@Slf4j
public abstract class AbstractParallelRetriever<T> {

    private final Executor executor;

    protected AbstractParallelRetriever(Executor executor) {
        this.executor = executor;
    }

    public final List<RetrievedChunk> executeParallelRetrieval(String question,
                                                                List<T> targets,
                                                                int topK) {

        // 局部 record: 把检索目标和 Future 绑定在一起
        record RetrievalFuture<T>(T target, CompletableFuture<List<RetrievedChunk>> future) {
        }

        // 1. 并行提交所有检索任务
        List<RetrievalFuture<T>> futures = targets.stream()
            .map(target -> {
                CompletableFuture<List<RetrievedChunk>> future = CompletableFuture.supplyAsync
                    () -> createRetrievalTask(question, target, topK),
                    executor
            });
        return new RetrievalFuture<>(target, future);
    })
        .toList();

        // 2. 等待完成并收集结果
        List<RetrievedChunk> allChunks = new ArrayList<>();
        int successCount = 0;
        int failureCount = 0;

        for (RetrievalFuture<T> future : futures) {
            try {
                List<RetrievedChunk> chunks = future.future.join();
                allChunks.addAll(chunks);
                successCount++;
            } catch (Exception e) {
                failureCount++;
                log.error("{} 获取检索结果失败 - 目标: {}",
                    getStatisticsName(), getTargetIdentifier(future.target), e);
            }
        }

        // 3. 统计日志
        log.info("{} 检索统计 - 总目标数: {}, 成功: {}, 失败: {}, 检索到 Chunk 总数: {}",
            getStatisticsName(), targets.size(), successCount, failureCount, allChunks.size())

        return allChunks;
    }

    // 三个抽象方法, 子类实现
    protected abstract List<RetrievedChunk> createRetrievalTask(String question, T target, int top
    protected abstract String getTargetIdentifier(T target);
    protected abstract String getStatisticsName();
}

```

几个设计细节值得注意:

final 修饰: 子类不能覆写 executeParallelRetrieval, 保证并行执行的骨架逻辑一致

泛型 <T>: 检索目标可以是任何类型——意图检索器的 T 是 IntentTask, Collection 检索器的 T 是 String

局部 record RetrievalFuture<T>: 把检索目标和对应的 Future 绑在一起。为什么不用 Pair? 局部 record 语义更清晰, 编译期类型安全, 而且只在这个方法内使用

future.join() 外层 try-catch: 单个检索任务失败只增加 failureCount, 不影响其他任务

3. 三个抽象方法

方法	职责	IntentParallelRetriever	CollectionParallelRetriever
createRetrievalTask	创建单个检索任务	从 IntentNode 取 collectionName 做检索	直接用 Collection 名称做检索
getTargetIdentifier	失败时打印目标标识	意图ID: xxx, 意图名称: xxx	Collection: xxx
getStatisticsName	统计日志的前缀	意图检索	全局检索

4. IntentParallelRetriever 的二次封装

意图检索器比 Collection 检索器复杂一些——每个意图有自己独立的 topK, 不能用父类统一的 topK 参数。所以它做了一个二次封装:

```
public class IntentParallelRetriever extends AbstractParallelRetriever<IntentParallelRetriever.IntentTask> {

    // 意图 + 预计算的 topK 绑定在一起
    public record IntentTask(NodeScore nodeScore, int intentTopK) {}

    // 四参数重载方法: 先计算每个意图的 topK, 再调父类
    public List<RetrievedChunk> executeParallelRetrieval(String question,
                                                         List<NodeScore> targets,
                                                         int fallbackTopK,
                                                         int topKMultiplier) {

        List<IntentTask> intentTasks = targets.stream()
            .map(nodeScore -> new IntentTask(
                nodeScore,
                resolveIntentTopK(nodeScore, fallbackTopK, topKMultiplier)
            ))
            .toList();
        return super.executeParallelRetrieval(question, intentTasks, fallbackTopK);
    }

    @Override
    protected List<RetrievedChunk> createRetrievalTask(String question, IntentTask task, int ignore) {
        NodeScore nodeScore = task.nodeScore();
        IntentNode node = nodeScore.getNode();
        try {
            return retrieverService.retrieve(
                RetrieveRequest.builder()
                    .collectionName(node.getCollectionName())
                    .query(question)
                    .topK(task.intentTopK()) // 用预计算的 topK
                    .build()
            );
        } catch (Exception e) {
            log.error("意图检索失败 - 意图ID: {}, Collection: {}",
                node.getId(), node.getCollectionName(), e);
            return List.of();
        }
    }

    // ... getTargetIdentifier, getStatisticsName
}
```

二次封装的思路是：先遍历所有意图，用 `resolveIntentTopK` 预先计算好每个意图的 `topK`（第 9 篇讲过——优先用节点配置 → 全局 fallback → 乘以倍率），封装成 `IntentTask` record。然后调父类的三参数方法 `executeParallelRetrieval`，父类只管并行提交和结果收集，不关心 `topK` 怎么来的。

注意 `createRetrievalTask` 的第三个参数 `ignoredTopK`——父类传入的统一 `topK` 在这里被忽略了，实际使用的是 `task.intentTopK()`。这是因为父类的模板方法设计时假设所有目标用同一个 `topK`，但意图检索的需求是每个目标独立 `topK`。通过 `IntentTask` record 提前绑定好各自的值，巧妙绕过了这个限制。

5. CollectionParallelRetriever 的极简实现

作为对比，Collection 检索器就简单多了：

```
public class CollectionParallelRetriever extends AbstractParallelRetriever<String> {

    private final RetrieverService retrieverService;

    public CollectionParallelRetriever(RetrieverService retrieverService, Executor executor) {
        super(executor);
        this.retrieverService = retrieverService;
    }

    @Override
    protected List<RetrievedChunk> createRetrievalTask(String question, String collectionName, int topK) {
        try {
            return retrieverService.retrieve(
                RetrieveRequest.builder()
                    .collectionName(collectionName)
                    .query(question)
                    .topK(topK)
                    .build()
            );
        } catch (Exception e) {
            log.error("在 collection {} 中检索失败", collectionName, e);
            return List.of();
        }
    }

    @Override
    protected String getTargetIdentifier(String collectionName) {
        return "Collection: " + collectionName;
    }

    @Override
    protected String getStatisticsName() {
        return "全局检索";
    }
}
```

泛型参数就是 `String`（Collection 名称），三个方法各两三行。不需要二次封装，直接调父类方法就行。

两个子类的对比体现了模板方法模式的价值：并行提交、等待、收集、容错、统计这些骨架逻辑写一次在父类里，子类只关心差异部分。`IntentParallelRetriever` 稍复杂（有 `topK` 预计算和 `IntentTask` 封装），`CollectionParallelRetriever` 极简，但并行执行和容错的逻辑完全复用。

容错设计：挂了怎么办

1. 两层容错机制

容错贯穿整个调用链，分为通道级和 Collection 级两层。

1.1 通道级容错

在 `executeSearchChannels` 中，每个 `Channel` 的 `search()` 调用被 `try-catch` 包裹：

```
        .map(channel -> CompletableFuture.supplyAsync(
            () -> {
                try {
                    return channel.search(context);
                } catch (Exception e) {
                    log.error("检索通道 {} 执行失败", channel.getName(), e);
                    return emptyResult(channel); // 返回空结果，不抛异常
                }
            },
            ragRetrievalExecutor
        ))
```

假设 IntentDirectedSearchChannel 的内部出了未预期的异常（比如数据库连接池耗尽导致 extractKbIntents 失败），它会返回一个空的 SearchChannelResult，不影响 VectorGlobalSearchChannel 正常执行。

1.2 Collection 级容错

在 AbstractParallelRetriever 里，两层 try-catch 做双重保险：

- 内层：createRetrievalTask 内部的 try-catch，单个 Collection 检索失败返回空列表，不抛异常
- 外层：executeParallelRetrieval 中 future.join() 外面的 try-catch，即使内层漏了异常，这里也能兜住

2. 一个具体的容错场景

用电场景跑一遍。用户问“售后怎么弄”，意图分数 0.55：

- 定向检索通道启用 (0.55 >= minIntentScore 0.4)，查 kb_after_sales
- 全局检索通道启用 (0.55 < confidenceThreshold 0.6)，扫全部 5 个 Collection
- 其中 kb_clothing_return 对应的向量数据库分区正在 compaction，检索超时

这时候会发生什么？

CollectionParallelRetriever 的 createRetrievalTask 里 try-catch 捕获异常，kb_clothing_return 返回空列表

统计日志打印：全局检索 检索统计 - 总目标数：5，成功：4，失败：1，检索到 Chunk 总数：12

MultiChannelRetrievalEngine 的统计日志：定向通道返回 4 个 Chunk (52ms)，全局通道返回 12 个 Chunk (少了服装退货的结果，65ms)，总共 16 个 Chunk 进入后处理器链

用户拿到的回答里没有服装退货的内容，但 3C、家电、售后维修的信息正常返回

容错的核心原则：宁可少返回一部分结果，也不让整个请求挂掉。少一个 Collection 的 Chunk 比返回 500 错误好得多。

上层视角：从 Pipeline 到向量库的完整调用链

1. 三层并行嵌套

把完整的调用链拉出来看：

```
StreamChatPipeline.retrieve()
→ RetrievalEngine.retrieve() // 子问题级并行 (ragContextExecutor)
→ MultiChannelRetrievalEngine.retrieveKnowledgeChannels() // 通道级并行 (ragRetrievalExecutor)
→ IntentDirectedSearchChannel.search()
→ IntentParallelRetriever.executeParallelRetrieval() // Collection 级并行 (ragInnerRetrievalExecutor)
→ RetrieverService.retrieve() // 单次 Milvus 向量检索
→ VectorGlobalSearchChannel.search()
→ CollectionParallelRetriever.executeParallelRetrieval()
→ RetrieverService.retrieve()
```

三层并行，三个独立的线程池：

层级	线程池	并行度

子问题级	ragContextExecutor	子问题数（通常 1~3）
通道级	ragRetrievalExecutor	启用的 Channel 数（通常 1~2）
Collection 级	ragInnerRetrievalExecutor	意图/Collection 数（通常 2~10）

三个池子完全隔离，互不阻塞。

2. 一次检索的全链路耗时

估算一下一次典型检索的全链路耗时（假设 8 核 CPU，两个通道同时启用）：

层级	并行度	单任务耗时	实际耗时
通道级	2 个通道并行	取最慢	~65ms
└ 定向检索	2 个 Collection 并行	~50ms	~52ms
└ 全局检索	5 个 Collection 并行	~50ms	~65ms
后处理器	串行	~100ms	~100ms
总计			~165ms

如果全部串行呢？2 个定向 Collection + 5 个全局 Collection，每个 50ms，共 350ms，加后处理 100ms，总计 450ms。并行省掉了约 63% 的检索延迟。

这里的后处理器串行执行，不是因为没法并行——去重和精排之间有依赖关系（先去重再精排），只能串行。第 11 篇会详细讲这条处理器链。

小结与下一篇预告

本篇拆解了多通道并行检索的内部实现，核心要点：

- 1. MultiChannelRetrievalEngine 分两阶段执行——阶段 1 并行执行所有启用的 Channel 收原始结果，阶段 2 串行执行后处理器链做精加工
- 2. SearchChannel 接口定义了五个方法（getName / getPriority / isEnabled / search / getType），通过 Spring 列表注入实现零改动扩展——加个 @Component 就自动生效
- 3. 两级线程池隔离解决嵌套并行的拥塞问题——外层 ragRetrievalExecutor（SynchronousQueue，Channel 级）和内层 ragInnerRetrievalExecutor（LinkedBlockingQueue(100)，Collection 级）
- 4. AbstractParallelRetriever 用模板方法模式封装并行执行骨架（final 方法），两个子类只需实现三个抽象方法：createRetrievalTask / getTargetIdentifier / getStatisticsName
- 5. IntentParallelRetriever 通过 IntentTask record 和四参数重载方法实现每个意图独立 topK 的预计算，CollectionParallelRetriever 则极简——泛型参数就是 String
- 6. 容错贯穿两层——通道级失败返回空结果，Collection 级失败返回空列表，都不会拖垮其他并行任务
- 7. TTL 包装确保子线程透传父线程的 ThreadLocal 上下文，链路追踪不断链
- 8. 三层并行嵌套（子问题 → 通道 → Collection）配合三个独立线程池，把串行 450ms 的检索压缩到 165ms

到这里，多个通道的原始结果已经收回来了。假设两个通道同时启用——定向检索从 kb_3c_warranty 和 kb_3c_return 各搜回了 8 条和 6 条，全局检索从 5 个 Collection 搜回了 15 条，加起来 29 条 Chunk。
29 条 Chunk 能直接塞进 LLM 的 Prompt 吗？不能。两个通道可能从同一个 Collection 搜到了同一条 Chunk（重复了），29 条里面质量参差不齐（有些和问题高度相关，有些只是沾点边），而生成阶段的 Prompt 最多放 5 条。怎么去重？怎么用 Cross-Encoder 精排？怎么截断到最终的 5 条？
第 11 篇讲——多个通道返回二三十条结果，最终只给模型 5 条。